

key_management

Key Management (1.0.0-MVP)

Field	Value
Revision	2
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Normative MVP specification for cryptographic key management: custody of the build-pipeline artifact-signing key, key rotation (overlap via <code>key_id</code>), the on-device trust store of signing public keys, and the forward path to an offline signing ceremony (HSM/KMS, threshold, timestamp re-signing) when TUF/Uptane lands in 1.0.1+ per ADR-0002. Routes signing/verify custody through the <code>security / Security-KMP</code> catalogue submodules. HelixConstitution clause numbers are UNVERIFIED against the authoritative text. The MVP has no threshold signing, no offline-key custody, and no signed revocation — the build-pipeline signing key is a single key whose compromise cannot be actively invalidated fleet-wide except via the §5 signed trust-store-update push, which is the highest MVP residual risk
Issues	(threat_model §171–182; ADR-0002 §5.2). The <code>security / Security-KMP</code> catalogue submodules' actual fit for key-custody / rotation / (future) threshold + offline-key primitives is UNVERIFIED (ADR-0002 §8 item 9; reuse map §3). The strength of Android-KeyStore device-key binding on the RK3588 / Orange Pi 5 Max board is UNVERIFIED. The concrete HSM/KMS product is not selected.

Field	Value
Fixed	Rev 2: committed the on-device trust-store UPDATE mechanism as MVP design (trust-store changes ship inside a signed OTA verified by the currently-trusted key; the only MVP rotation/recovery path) (§5, §9); sharpened “rotation is not revocation” — a compromised key cannot be actively invalidated fleet-wide except via that trust-store push, marked as the highest MVP residual risk (§4, §9); reflected the ED25519 DEFAULT signature scheme with key_id agility from signing_verification.md §3 (§2).
Continuation	Inspect security / Security-KMP to confirm they host key custody + rotation (and can later host threshold/offline-key primitives) and remove UNVERIFIED tags; define and dry-run the offline signing-ceremony runbook before any TUF root/targets keys are generated (ADR-0002 §4.3 step 3); select the HSM/KMS; produce the mTLS/device-cert provisioning plan if mTLS is adopted in 1.0.1+ (see transport_security.md §6).

Table of contents

1. Purpose and scope
2. Key inventory (MVP)
3. Custody of the build-pipeline signing key
4. Key rotation
5. On-device trust store
6. Forward path: offline signing ceremony (TUF/Uptane, 1.0.1+)
7. Catalogue reuse and module boundaries
8. Testing (four-layer)
9. Open / UNVERIFIED items
10. Compliance notes (HelixConstitution)
11. Sources

The table-of-contents requirement is mandated by HelixConstitution §11.4.61 (UNVERIFIED clause number). This document carries its ToC immediately after the metadata table.

1. Purpose and scope

This document specifies how Helix OTA manages cryptographic keys in the 1.0.0-MVP: where the **build-pipeline artifact-signing key** lives and who can use it (custody), how it is **rotated** without bricking the fleet, what the **on-device trust store** holds and how it is updated, and the **forward path** to a hardened offline signing ceremony (threshold, HSM/KMS, timestamp re-signing) when TUF/Uptane lands in 1.0.1+ per ADR-0002.

It is the companion to [signing_verification.md](#) (which uses the keys) and [transport_security.md](#) (which manages the *separate* TLS certificate trust). It honors, and does not re-decide, the LOCKED strategy: signing + SHA-256 + AVB for MVP; TUF device-side deferred to 1.0.1 per ADR-0002. Key custody is routed through the verified catalogue (security, Security-KMP); no bespoke crypto/key brick is invented (ADR-0002 §4.2).

2. Key inventory (MVP)

Three distinct trust anchors exist in the MVP; they are managed and rotated **independently** so a compromise of one does not cascade:

Key / anchor	Type	Held by	Verified against	Specified in
Build-pipeline artifact-signing key	Asymmetric private (MVP DEFAULT = ED25519 , with key_id-based algorithm agility for ECDSA-P256 / RSA-PSS fallback, see signing_verification.md §3)	Build pipeline (not the online control plane)	Public key in the server trust store (upload) + on-device trust store (apply)	This document §3–§5
TLS server certificate(s)	X.509 (TLS 1.3)	Control-plane edge	Device/client TLS trust roots	transport_security.md §5
Device-identity key/token	Hardware-bound (Android KeyStore)	Each device	auth (server) via the device token	master §6; threat_model §4.7; this doc §5 (relationship only)

The **critical separation**: the artifact-signing key proves *which firmware is authentic*; the TLS certificate proves *who the server is*; the device-identity key proves *which device is calling*. Compromise of the TLS cert or a device token does **not** allow artifact forgery, because artifacts are verified against the *signing* trust store on-device ([signing_verification.md §6](#); threat_model §4.6).

3. Custody of the build-pipeline signing key

- **The signing key is the build-pipeline key, not an online-control-plane key** (master §6; threat_model §171–175). The online Go control plane verifies signatures (it holds **public** keys) but never holds the **private** signing key, so popping the control plane does not yield signing capability. This is the primary MVP blast-radius control (threat_model §171–175).
- **Custody routing:** key custody is routed through the security catalogue submodule (server side) and Security-KMP (device side) rather than a bespoke key brick (ADR-0002 §4.2; reuse map §3). That these submodules expose key custody/rotation primitives is **UNVERIFIED** (ADR-0002 §8 item 9) and must be confirmed before implementation (Continuation).
- **Use through the signer abstraction.** The private key is used only via the signer abstraction (compatible with go-securesystemslib signature.Signer; [signing_verification.md](#) §4, §8), so the same custody seam serves the MVP detached-signature signer and a future TUF role signer.
- **MVP custody posture (accepted residual):** the MVP has **no threshold signing and no offline-key custody** — it is a single online-usable signing key in the build pipeline (threat_model §179–182; ADR-0002 §5.2). A stolen key is a fleet-wide compromise until rotation, and rotation in the MVP has limited recovery semantics (no signed key-revocation metadata; that is a TUF property deferred to 1.0.1+). This is tracked, not solved, in the MVP. The hardening path is §6.
- **Access control + audit:** access to the signing key is restricted to the build pipeline's signing step; every release/sign and every admin publish action is audit-logged (master §6).

4. Key rotation

The MVP supports **rotation with overlap** so a new signing key can be introduced before the old one is retired, avoiding a flag-day:

- **key_id selects the verifying key.** Each signed release carries a key_id ([signing_verification.md](#) §4); the server and device trust stores can hold **multiple** active public keys, and verification selects the one named by key_id. This permits an **overlap window**: releases signed by the new key verify on devices that already trust it, while in-flight releases signed by the old key still verify.
- **Rotation procedure (MVP):**
 1. Generate the new signing keypair (custody per §3).
 2. **Distribute the new public key to the device trust store first** (§5) and to the server trust store, so both new and old key_ids verify (overlap begins).
 3. Switch the build pipeline to sign with the new key (key_id = new).
 4. After the fleet has converged on the new public key and no old-key releases remain in flight, **retire the old public key** from both trust stores (overlap ends).
- **Rotation is not revocation (highest MVP residual risk).** Rotation *retires* an old key from trust stores on the next trust-store push; it does **not** actively invalidate a key. **Explicitly: a compromised signing key cannot be actively invalidated fleet-wide in the MVP except via a trust-store-update push (§5) that removes it** — there is **no signed, freshness-checked revocation metadata** that would let a device reject the compromised key on its own before that push

reaches it (that is a TUF key-compromise-recovery property deferred to 1.0.1+; ADR-0002 §3.2, §5; threat_model §179–182). Until the trust-store-removal push lands on a given device, that device will still trust artifacts signed by the compromised key. This is the **highest residual risk of the MVP key posture**: the recovery window is bounded only by how fast the signed trust-store update converges across the fleet. Emergency rotation therefore depends entirely on the §5 trust-store-update push, which is itself the gating mechanism and must be integrity-protected.

- **TLS certificate rotation and device-token rotation** are separate processes ([transport_security.md §5](#); master §6) and do not interact with signing-key rotation.

5. On-device trust store

- **Contents**: the device trust store holds the **build-pipeline signing public key(s)** used to verify artifact signatures before apply ([signing_verification.md §6](#); master §6). It is distinct from the TLS trust roots ([transport_security.md §5](#)) and from the device-identity key (below).
- **Initial provisioning**: the trust store ships with the device build (the active build-pipeline public key is embedded), so a freshly imaged device can verify the first OTA. Device transport/trust config is carried via Config-KMP (master §6); the signing public keys themselves are consumed by the Security-KMP verify path.
- **Update mechanism (COMMITTED MVP design)**: introducing or retiring a signing public key (§4) requires updating the on-device trust store. The **MVP design is committed, not deferred**: trust-store changes **ship inside a signed OTA that is verified by the currently-trusted signing key** (the existing device re-verify-before-apply gate; [signing_verification.md §6](#)) before any new key becomes authoritative. A trust-store update must itself be authenticated — otherwise it becomes the new single point of forgery — and binding it to a signed OTA reuses the exact authenticity gate already enforced for every artifact, adding no new trust root. **Because the MVP has no signed revocation metadata until TUF (§4, 1.0.1+), this OTA-delivered trust-store push is the only rotation/recovery path, so it must be specified now rather than left open.** It is therefore specified here as the committed MVP mechanism. (The security/Security-KMP primitives that apply the trust-store change on-device remain a tracked confirm-against-the-brick item — §7, §9 — but the delivery design itself is decided.)
- **Relationship to device identity**: the device-identity key (Android KeyStore, hardware-bound token; master §6; threat_model §4.7) is **separate** from the signing trust store — it authenticates the device *to the server*, not artifacts *to the device*. Its binding strength on the RK3588 / Orange Pi 5 Max board is **UNVERIFIED** (threat_model §4.7).
- **Fail-safe**: a device that cannot verify an artifact against any trusted key **MUST** abort the apply (never apply on verification failure; [signing_verification.md §6](#)); native A/B guarantees the running slot is untouched.

6. Forward path: offline signing ceremony (TUF/Uptane, 1.0.1+)

The MVP key posture (single online-usable key, no offline custody, no threshold) is deliberately the *floor*; ADR-0002 §4.3 owns the hardening sequence. This section is the forward-path specification, not a 1.0.0 deliverable.

- **Why it is needed:** plain signing collapses to a single key; TUF adds **thresholds + offline keys** to survive key compromise and adds signed, freshness-checked metadata (rollback/freeze/mix-and-match/ mirror-denial resilience) — the attack classes the MVP leaves open (ADR-0002 §3.1, §3.2, §5; threat_model §179–182).
- **Offline key-custody ceremony (ADR-0002 §4.3 step 3):** define and dry-run a runbook covering:
 - **Root / targets keys held offline** (HSM or KMS-backed), brought online only for signing ceremonies; the specific HSM/KMS product is **not selected** (Continuation).
 - **Threshold signing** for the high-value roles (root/targets), so no single key/operator can mint a release (ADR-0002 §3.2 thresholds + offline keys).
 - **Timestamp re-signing automation** — the TUF timestamp (and snapshot) roles re-sign on a short cadence and use **online** keys, separate from the offline root/targets keys (ADR-0002 §3.2, §4.3).
 - **Rotation + revocation** semantics that TUF provides (delegations, key-compromise recovery), replacing the MVP's limited retire-from-trust-store mechanism (§4).
- **MVP-forward seams that make this drop-in (no rework):** the signer abstraction (one seam for MVP + TUF role signers), opaque target identity (path+length+sha256 layers under a TUF targets entry byte-identically), the distinct verify-gate (a TUF refresh/verify flow inserts ahead of the existing check), and the reserved per-device target decision (future Director repo) ([signing_verification.md §8](#); ADR-0002 §4.2).
- **Gating:** no on-device TUF enforcement (and thus no mandatory new key roles on-device) is made mandatory until the ADR-0002 §4.3 spikes close — especially the **on-device client spike** (the dominant, UNVERIFIED cost) and the **key-custody ceremony** definition (ADR-0002 §4.3, §6).

7. Catalogue reuse and module boundaries

Per catalogue-first reuse (§11.4.74, UNVERIFIED clause); no bespoke key/crypto brick is invented (ADR-0002 §4.2):

Concern	Submodule(s)	Class	Boundary
Server-side signing-key custody + signature primitives	security	reuse	Key custody + crypto primitives behind the signer abstraction. UNVERIFIED that security hosts custody/rotation (and later threshold/offline-key) primitives (ADR-0002 §8 item 9).

Concern	Submodule(s)	Class	Boundary
Device-side verify-key handling + crypto	Security-KMP	reuse	On-device trust-store verify path. UNVERIFIED surface.
Device trust-root / signing-public-key config delivery	Config-KMP	reuse	Carries device config incl. trust material; the trust-store <i>update</i> mechanism itself must be authenticated (§5).
Device-identity key (hardware-bound token)	Auth-KMP, Security-KMP; server: auth	reuse	Authenticates device to server (separate from signing trust).
TLS certificate trust	http3, Config-KMP	reuse	Separate anchor; see transport_security.md §5 .
Signer abstraction (signature.Signer seam)	consumed by ota-artifact-validator / build signer	new (consumer)	One seam for MVP detached-sig + future TUF role signers (signing_verification.md §8).

8. Testing (four-layer)

Per HelixConstitution §1 (UNVERIFIED clause); key handling sits on the $\geq 90\%$ safety-critical signing/verify path (master §13), with no-bluff positive evidence (§7.1):

- **Layer 1 — Source-presence gate.** Static check that: the control plane holds **only public** verify keys (no signing private key in the online control-plane source); signing goes through the signer abstraction (not raw key use); verification selects the key by `key_id`; trust stores support **multiple active keys** (overlap); and a verification failure aborts apply. Absence is a build-time failure.
- **Layer 2 — Artifact gate (bytes shipped).** Confirm the device build embeds a non-empty signing trust store with the active public key; confirm no signing **private** key is present in any shipped server/device artifact; confirm each released artifact carries a `key_id` resolvable in the trust store.
- **Layer 3 — Runtime / integration.** Exercise rotation end-to-end: (a) distribute new public key $\rightarrow$ both `old-key_id` and `new-key_id` releases verify (overlap); (b) switch signing to the new key $\rightarrow$ new releases verify on updated devices; (c) retire old key $\rightarrow$ `old-key_id` releases no longer verify. **Negative cases:** a release whose `key_id` is unknown/retired $\rightarrow$ reject; a release signed by a key absent from the trust store $\rightarrow$ reject; an attempt to apply on verification failure $\rightarrow$ abort (A/B slot untouched).
- **Layer 4 — Mutation meta-test (PASS $\rightarrow$ FAIL on negation).** Mutate key handling and require PASS $\rightarrow$ FAIL: accept any `key_id` regardless of trust store, skip the `key_id` lookup, treat a retired key as still trusted, or apply despite a verify failure — each **MUST** be caught by Layer 3. A surviving mutant is a defect on a safety-critical path.

The offline-ceremony / HSM-KMS / threshold paths (§6) are **not** MVP-testable here; their validation is the ADR-0002 §4.3 step-3 ceremony dry-run, which is the rock-solid-proof closure for those forward items.

9. Open / UNVERIFIED items

1. **security / Security-KMP host key custody + rotation** (and can later host threshold/offline-key primitives) — **UNVERIFIED** (ADR-0002 §8 item 9; reuse map §3).
2. **Signature scheme: DEFAULT pinned (ED25519)**, with key_id-based algorithm agility for ECDSA-P256 / RSA-PSS fallback — decided in [signing_verification.md §3](#); only the confirm-against-the-security-brick (primitives exposed) remains tracked there.
3. **On-device trust-store update mechanism — COMMITTED MVP design** (trust-store changes ship inside a signed OTA verified by the currently-trusted key; the only MVP rotation/recovery path) (§5). Only the security/Security-KMP on-device apply primitive remains a confirm-against-the-brick item.
4. **No threshold signing, no offline-key custody, no signed revocation in the MVP** — single-signing-key exposure is an accepted MVP residual until 1.0.1+ TUF; a compromised key can only be invalidated fleet-wide via the §5 trust-store-update push, the **highest MVP residual risk** (ADR-0002 §5.2; threat_model §179–182).
5. **HSM/KMS product not selected**; offline signing-ceremony runbook not yet defined/dry-run (ADR-0002 §4.3 step 3) — forward item.
6. **Android-KeyStore device-key binding strength on RK3588 / Orange Pi 5 Max** — **UNVERIFIED** (threat_model §4.7).
7. **HelixConstitution clause numbers** — **UNVERIFIED** (corpus convention).

10. Compliance notes (HelixConstitution)

Clause (UNVERIFIED numbers)	How this spec complies
§11.4.61 (ToC)	ToC present immediately after the metadata table.
§7.1 / §11.4.6 (no-bluff / no-guessing)	Every claim cites an ADR, master §6, or the threat model; the signature-scheme DEFAULT (ED25519) and the trust-store-update mechanism are now decided MVP design with only their security/Security-KMP primitive fit carried as UNVERIFIED ; submodule custody fit, HSM/KMS, and KeyStore binding remain UNVERIFIED , not asserted. The MVP single-key residual — and that a compromised key is only recoverable via the §5 trust-store push — is stated, not hidden.
§11.4.74 (catalogue-first reuse)	Key custody routed through security / Security-KMP / Config-KMP / Auth-KMP; no bespoke key brick invented (§7).

Clause (UNVERIFIED numbers)	How this spec complies
§11.4.28 (decoupling)	Three trust anchors managed independently; signing key used only via the signer abstraction seam (§2, §3, §7).
§1 / §1.1 (four-layer + mutation)	§8 specifies all four layers, including rotation-overlap and verify-failure-abort mutations.
§11.4.123 (rock-solid proof)	MVP claims closed by Layer-3 rotation evidence; forward (offline-ceremony/threshold) items closed by the ADR-0002 §4.3 step-3 ceremony dry-run before any TUF keys are generated, not by assertion.
§11.4.125 (code-review gate)	Subject to the mandatory adversarial code-review subagent before acceptance (master §14).

11. Sources

All paths relative to docs/research/main_specs/:

- [research/adr/adr-0002-supply-chain-trust.md](#) — §3.1–§3.2 (attack classes, thresholds/offline keys), §4.1–§4.3 (MVP trust, MVP-forward interface, adoption sequencing incl. key-custody ceremony), §5.2 (single-key negative), §8 (UNVERIFIED items).
- [00-master/2026-06-07-helix-ota-design.md](#) — §6 (build-pipeline key signs; public key in device trust store; device identity; forward path), §13 (four-layer testing).
- [00-master/threat_model.md](#) — signing-key asset + single-key residual (lines ~90, 108, 154–182), §4.7 (device impersonation / KeyStore binding).
- [00-master/submodule_reuse_map.md](#) — §3 (artifact intake/validation, Android client), §5 (security/Security-KMP device-identity upstream addition).
- [00-master/documentation_standards.md](#) — §2 (metadata), §8 (anti-bluff), §9 (canonical submodules).
- [signing_verification.md](#), [transport_security.md](#) — companion MVP security specs.